Credit Risk & Loan Default Analysis

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

plt.rcParams["figure.figsize"] = (8,5)
# plt.rcParams["figure.dpi"] = 100
```

```
!gdown 152xpqGOH3aWStqebrCNT8Kj-2wCXkjxZ
```

```
Downloading...
From: https://drive.google.com/uc?id=152xpqGOH3aWStqebrCNT8Kj-2wCXkjxZ
To: /content/Credit_risk_prediction_dataset.csv
100% 298k/298k [00:00<00:00, 74.0MB/s]
```

```
df = pd.read_csv("Credit_risk_prediction_dataset.csv")
```

```
df.head()
```

	Age	Income	LoanAmount	CreditScore	YearsExperience	Gender	Education	City	EmploymentType	LoanApproved
0	56	48353.0	31258	675.0	20	Female	High School	Houston	Unemployed	0
1	69	57462.0	23262	586.0	6	Male	High School	San Francisco	Self-Employed	0
2	46	44219.0	26530	781.0	26	Male	PhD	Houston	Self-Employed	1
3	32	56307.0	11531	549.0	11	Male	NaN	New York	Unemployed	0

Next steps: Generate code with df New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 10 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Age              5000 non-null   int64
 1   Income           4804 non-null   float64
 2   LoanAmount       5000 non-null   int64
 3   CreditScore      4806 non-null   float64
 4   YearsExperience  5000 non-null   int64
 5   Gender           5000 non-null   object
 6   Education        4802 non-null   object
 7   City             5000 non-null   object
 8   EmploymentType   5000 non-null   object
 9   LoanApproved     5000 non-null   int64
dtypes: float64(2), int64(4), object(4)
memory usage: 390.8+ KB
```

```
df.describe()
```

	Age	Income	LoanAmount	CreditScore	YearsExperience	LoanApproved
count	5000.000000	4804.000000	5000.000000	4806.000000	5000.000000	5000.000000
mean	43.584600	49738.123022	19870.768600	575.494590	19.599000	0.230200
std	14.919094	15101.361851	8046.542413	160.550839	11.516837	0.421003
min	18.000000	-3731.000000	-10059.000000	300.000000	0.000000	0.000000
25%	31.000000	39608.500000	14455.250000	433.000000	10.000000	0.000000
50%	43.000000	49488.000000	19842.500000	579.000000	20.000000	0.000000
75%	56.000000	59917.000000	25326.750000	712.000000	29.000000	0.000000
max	69.000000	99146.000000	48353.000000	849.000000	39.000000	1.000000

Filling the missing value

```
df['Income'] = df['Income'].fillna(df['Income'].mean())

df['CreditScore'] = df['CreditScore'].fillna(df['CreditScore'].mean())
```

Replacing missing values on education with each emloyeement type

```
df['Education'] = df['Education'].fillna(
    df.groupby('EmploymentType')['Education'].transform(lambda x: x.mode()[0])
)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 10 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Age              5000 non-null   int64
 1   Income           5000 non-null   float64
 2   LoanAmount       5000 non-null   int64
 3   CreditScore      5000 non-null   float64
 4   YearsExperience  5000 non-null   int64
 5   Gender           5000 non-null   object
 6   Education        5000 non-null   object
 7   City             5000 non-null   object
 8   EmploymentType   5000 non-null   object
 9   LoanApproved     5000 non-null   int64
dtypes: float64(2), int64(4), object(4)
memory usage: 390.8+ KB
```

```
numeric_cols = df.select_dtypes(include="number").columns
object_cols = df.select_dtypes(include="object").columns

print("🔢 NUMERIC COLUMNS")
print("-" * 50)

for col in numeric_cols:
    nunique = df[col].nunique()
    print(f"\n📌 {col}")
    print(f"nunique: {nunique}")

    if nunique < 15:
        print("unique values:", df[col].unique())

print("\n" + "=" * 60)

print("🔤 OBJECT COLUMNS")
print("-" * 50)

for col in object_cols:
    nunique = df[col].nunique()
    print(f"\n📌 {col}")
    print(f"nunique: {nunique}")

    if nunique < 15:
        print("unique values:", df[col].unique())
```

```
🔢 NUMERIC COLUMNS
--------------------------------------------------

📌 Age
nunique: 52

📌 Income
nunique: 4600

📌 LoanAmount
nunique: 4588

📌 CreditScore
nunique: 551

📌 YearsExperience
nunique: 40

📌 LoanApproved
nunique: 2
unique values: [0 1]

============================================================
```

```
🔤 OBJECT COLUMNS
--------------------------------------------------

📌 Gender
nunique: 2
unique values: ['Female' 'Male']

📌 Education
nunique: 4
unique values: ['High School' 'PhD' 'Masters' 'Bachelors']

📌 City
nunique: 4
unique values: ['Houston' 'San Francisco' 'New York' 'Chicago']

📌 EmploymentType
nunique: 3
unique values: ['Unemployed' 'Self-Employed' 'Salaried']
```

```
df.head()
```

	Age	Income	LoanAmount	CreditScore	YearsExperience	Gender	Education	City	EmploymentType	LoanApproved
0	56	48353.0	31258	675.0	20	Female	High School	Houston	Unemployed	0
1	69	57462.0	23262	586.0	6	Male	High School	San Francisco	Self-Employed	0
2	46	44219.0	26530	781.0	26	Male	PhD	Houston	Self-Employed	1
3	32	56307.0	11531	549.0	11	Male	Masters	New York	Unemployed	0

Next steps: Generate code with df New interactive sheet

Adding custome columns

```
df['loan_staus']  = df['LoanApproved'].map({ 1 :'loan approved' , 0 :'loan rejected' })
```

```
df['Income_Bin'] = pd.cut(
    df['Income'],
    bins=[0, 30000, 60000, 90000, float('inf')],
    labels=['Low', 'Medium', 'High', 'Very High']
)
```

```
df['LoanAmount_Bin'] = pd.cut(
    df['LoanAmount'],
    bins=[0, 10000, 20000, 30000, float('inf')],
    labels=['Small', 'Medium', 'Large', 'Very Large']
)
```

```
df['CreditScore_Bin'] = pd.cut(
    df['CreditScore'],
    bins=[0, 580, 670, 740, 800, float('inf')],
    labels=['Poor', 'Fair', 'Good', 'Very Good', 'Excellent']
)
```

```
df['CreditScore_Bin'] = pd.cut(
    df['CreditScore'],
    bins=[0, 580, 670, 740, 800, float('inf')],
    labels=['Poor', 'Fair', 'Good', 'Very Good', 'Excellent']
)
```

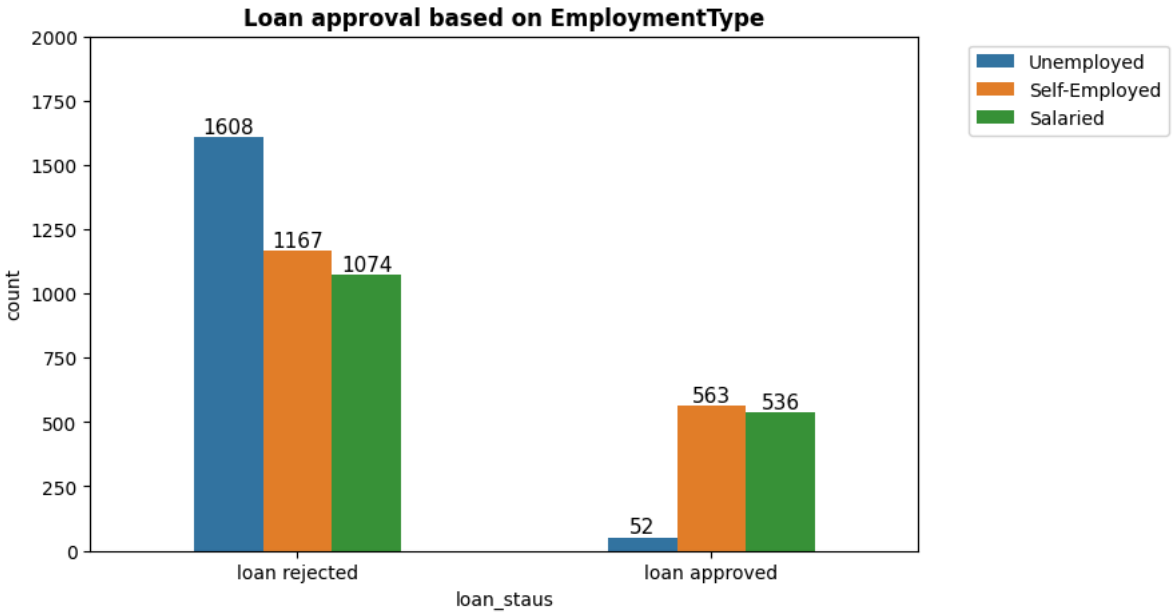
```
df['Experience_Bin'] = pd.cut(
    df['YearsExperience'],
    bins=[0, 5, 10, 20, float('inf')],
    labels=['0-5 yrs', '5-10 yrs', '10-20 yrs', '20+ yrs']
)
```

Data Visulisation and analytics

Loan Approval Rate based on employment type

```
ax = sns.countplot( data = df,  x = 'loan staus',  hue = 'EmploymentType',  width= 0.5)
```

```
plt.legend ( bbox_to_anchor = (1.05,1)  , loc = 'upper left' )
plt.ylim( 0 , 2000)
for i in ax.containers:
  ax.bar_label( i , fontsize = 11 )
plt.title('Loan approval based on EmploymentType' , fontweight='bold')
plt.show()
```

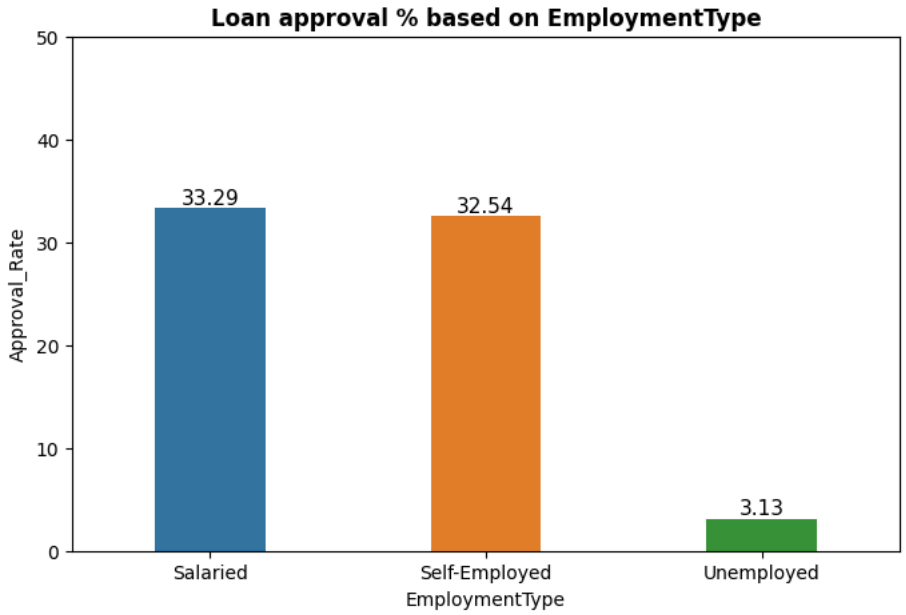



```
ratio_df = df.groupby('EmploymentType')['LoanApproved'].agg(['count','sum'])

ratio_df['Approval_Rate'] = ratio_df['sum'] / ratio_df['count']
ratio_df['Rejection_Rate'] = 1 - ratio_df['Approval_Rate']

ratio_df['Approval_Rate'] = (ratio_df['Approval_Rate'] * 100).round(2)
ratio_df['Rejection_Rate'] = (ratio_df['Rejection_Rate'] * 100).round(2)

ratio_df = ratio_df.reset_index()

ratio_df
```

	EmploymentType	count	sum	Approval_Rate	Rejection_Rate
0	Salaried	1610	536	33.29	66.71
1	Self-Employed	1730	563	32.54	67.46
2	Unemployed	1660	52	3.13	96.87

Next steps: Generate code with ratio_df New interactive sheet

```
ax =sns.barplot( data = ratio_df , y = 'Approval_Rate' , x = 'EmploymentType' , hue = 'EmploymentType' , width = 0.4)
plt.ylim(0 , 50)
for i in ax.containers:
  ax.bar_label( i , fontsize = 11 )
plt.title('Loan approval % based on EmploymentType' , fontweight='bold')
plt.show()
```



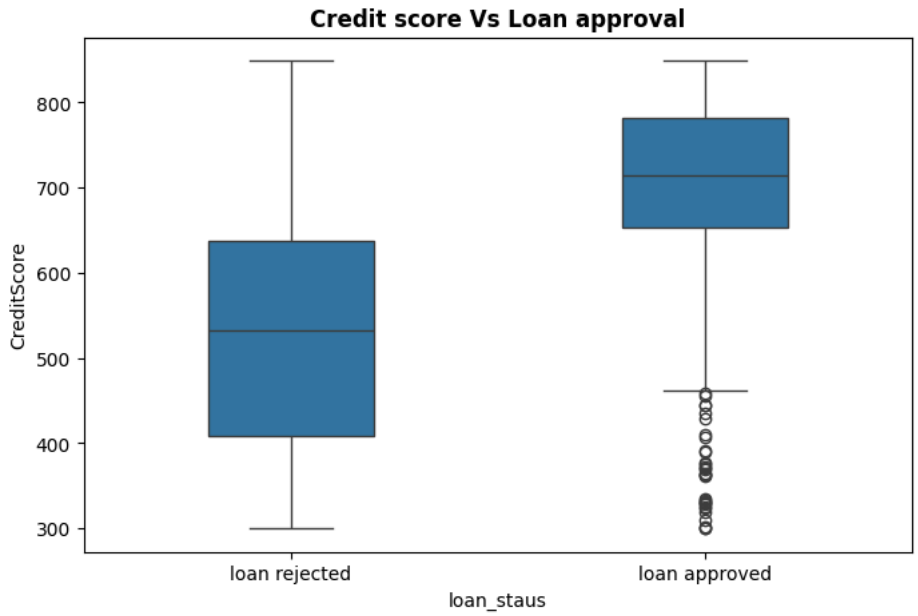
1. Salaried applicants have the **highest approval rate (~33.29%)**
2. Self-employed are **slightly lower (~32.54%)**
3. Unemployed applicants have **extremely low approval (~3.13%)**

Employment status is a strong primary filter, with employed applicants showing ~10x higher approval probability

Approval rates are nearly identical for salaried and self-employed applicants, indicating that income stability matters more than job type

Credit score Vs Loan approval

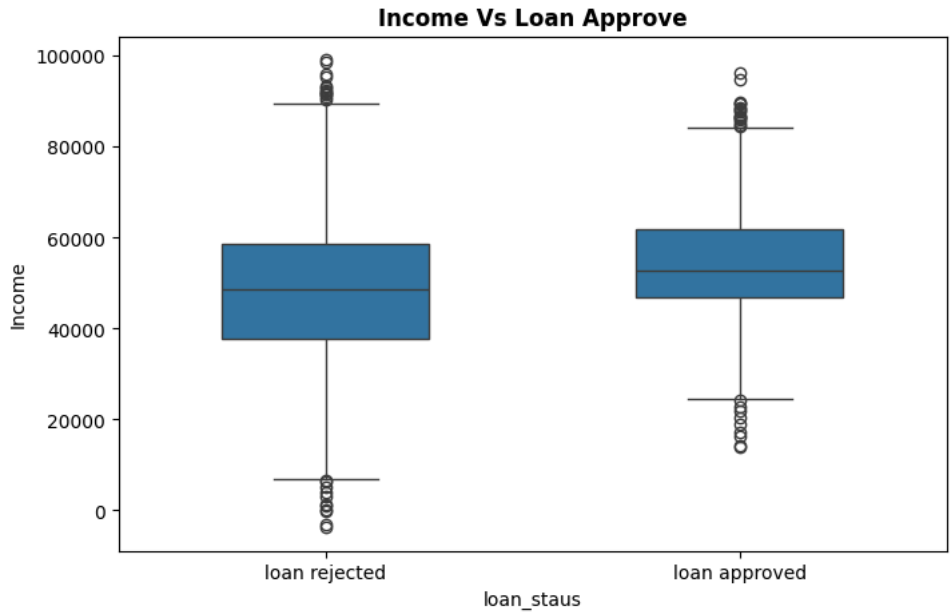
```
sns.boxplot( data = df , x = 'loan_staus' , y = 'CreditScore' , width= 0.4 )
plt.title('Credit score Vs Loan approval' , fontweight='bold')
plt.show()
```



The box plot clearly shows that approved applicants have significantly higher and more consistent credit scores, while rejected applicants are concentrated in lower score ranges, with a few exceptions indicating influence of additional factors

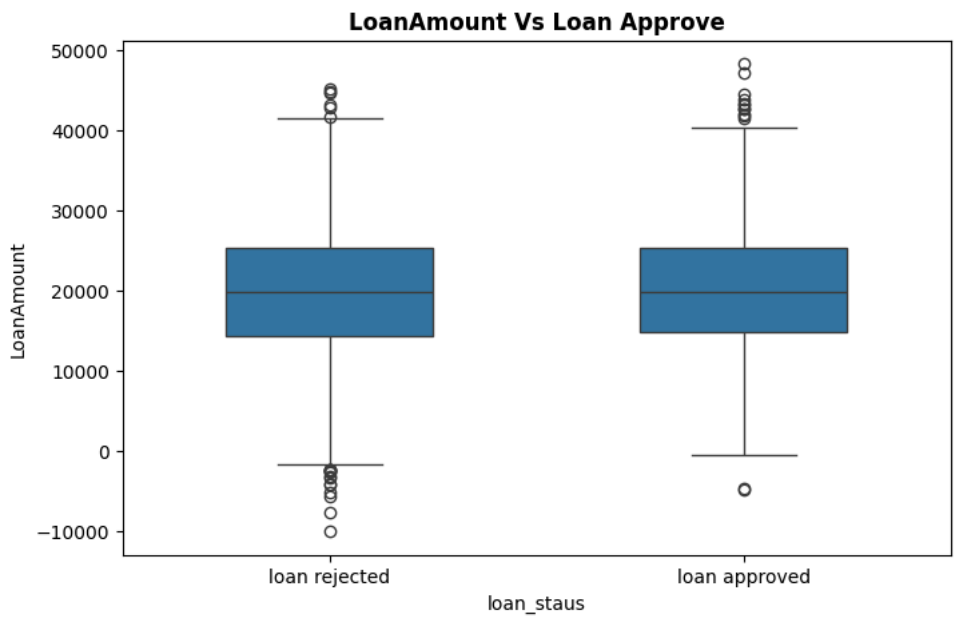
Credit score shows a strong threshold behavior, with most approvals occurring above ~650, making it the most influential feature in loan decisions

```
sns.boxplot( data = df ,  y = 'Income' , x = 'loan_staus' , width= 0.5 )
plt.title("Income Vs Loan Approve"  , fontweight='bold')
plt.show()
```



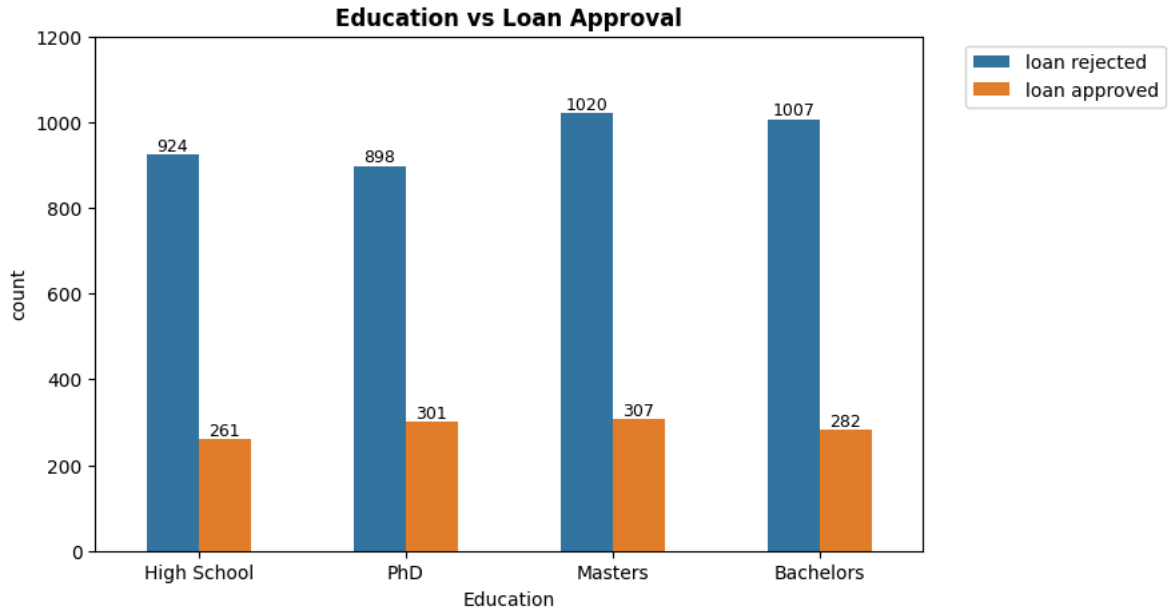
Income has a moderate impact on loan approval, but significant overlap between approved and rejected applicants indicates it is not a decisive factor

```
sns.boxplot( data = df ,  y = 'LoanAmount' , x = 'loan_staus' , width = 0.5 )
plt.title("LoanAmount Vs Loan Approve"  , fontweight='bold')
plt.show()
```



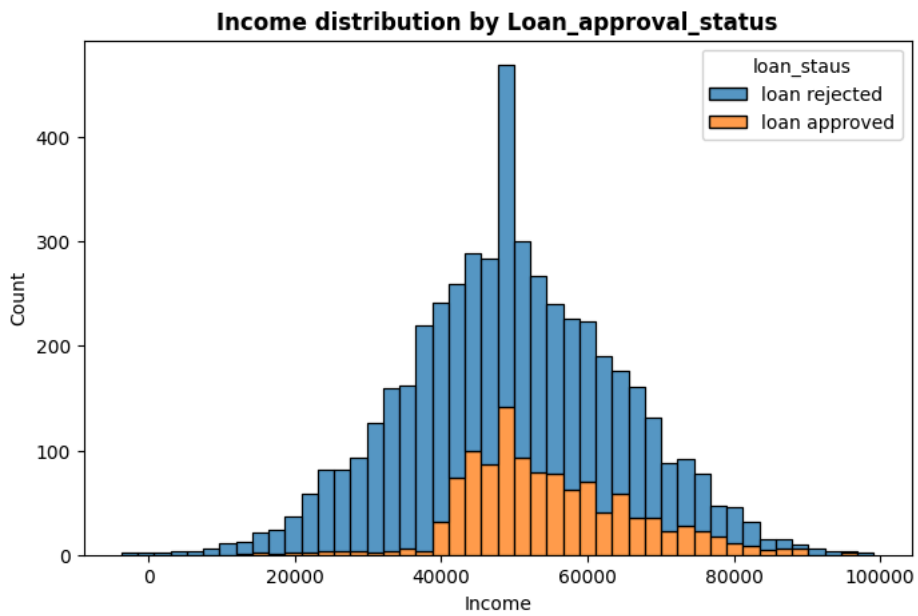
Loan amount has minimal influence on approval decisions, as both approved and rejected applicants show similar distributions

```
ax =sns.countplot( data = df , x = 'Education' , hue = 'loan_staus' , width = 0.5 )
plt.legend ( bbox_to_anchor = (1.04,1)  , loc = 'upper left' )
for container in ax.containers:
    ax.bar_label(container , fontsize=9 )
plt.title("Education vs Loan Approval"  , fontweight='bold' )
plt.ylim(0, 1200)
plt.show()
```



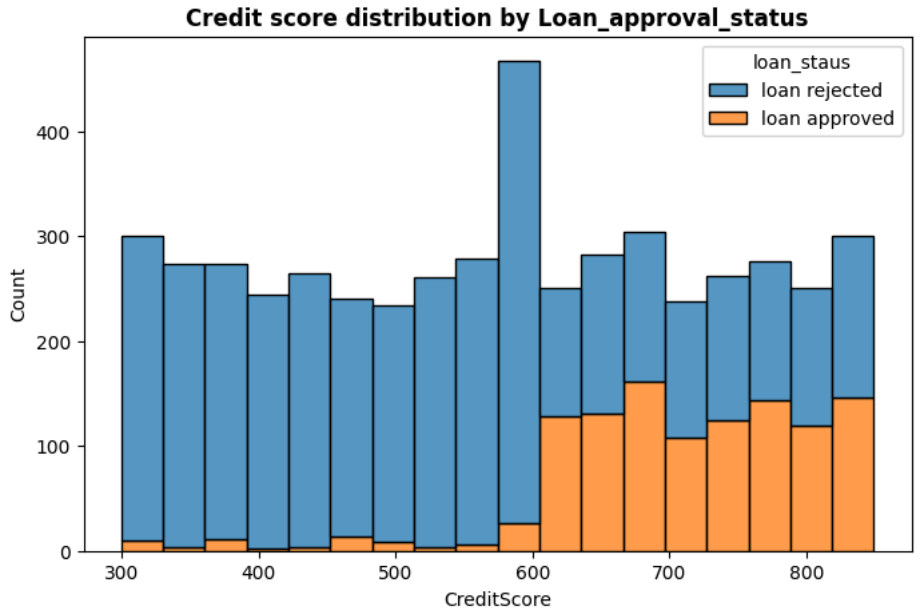
Education level shows limited impact on loan approval, with similar approval counts across categories, suggesting it is not a key decision factor

```
sns.histplot( data = df , x = 'Income' , hue = 'loan_staus' , multiple = 'stack')
plt.title( 'Income distribution by Loan_approval_status' ,  fontweight = 'bold')
plt.show()
```



Approved applicants are slightly concentrated in mid-to-high income ranges, but overlap with rejected applicants indicates income alone is insufficient for approval

```
sns.histplot( data = df , x = 'CreditScore' , hue = 'loan_staus' , multiple = 'stack' )
plt.title( 'Credit score distribution by Loan_approval_status' ,  fontweight = 'bold')
plt.show()
```

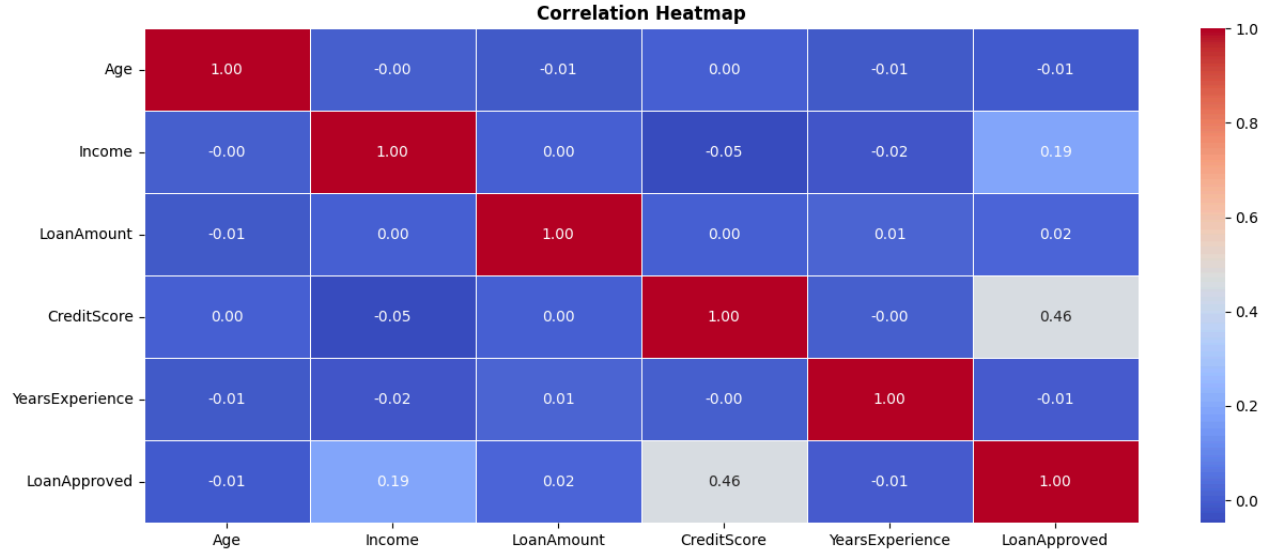


Approved applicants are concentrated in higher credit score ranges (650+), reinforcing credit score as the dominant approval factor

```
corr = df.corr(numeric_only=True)
corr.round(2)
```

	Age	Income	LoanAmount	CreditScore	YearsExperience	LoanApproved
Age	1.00	-0.00	-0.01	0.00	-0.01	-0.01
Income	-0.00	1.00	0.00	-0.05	-0.02	0.19
LoanAmount	-0.01	0.00	1.00	0.00	0.01	0.02
CreditScore	0.00	-0.05	0.00	1.00	-0.00	0.46
YearsExperience	-0.01	-0.02	0.01	-0.00	1.00	-0.01
LoanApproved	-0.01	0.19	0.02	0.46	-0.01	1.00

```
plt.figure(figsize=(15,6))
sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    linewidths=0.5
)
plt.title("Correlation Heatmap" , fontweight='bold')
plt.show()
```



- CreditScore → 0.46
- Income → 0.19
- LoanAmount → 0.02

Correlation analysis confirms that credit score has the strongest relationship with loan approval, while loan amount has negligible impact

```
df.head()
```

	Age	Income	LoanAmount	CreditScore	YearsExperience	Gender	Education	City	EmploymentType	LoanApproved	loan_st
0	56	48353.0	31258	675.0	20	Female	High School	Houston	Unemployed	0	I rejec
1	69	57462.0	23262	586.0	6	Male	High School	San Francisco	Self-Employed	0	I rejec
2	46	44219.0	26530	781.0	26	Male	PhD	Houston	Self-Employed	1	I appro

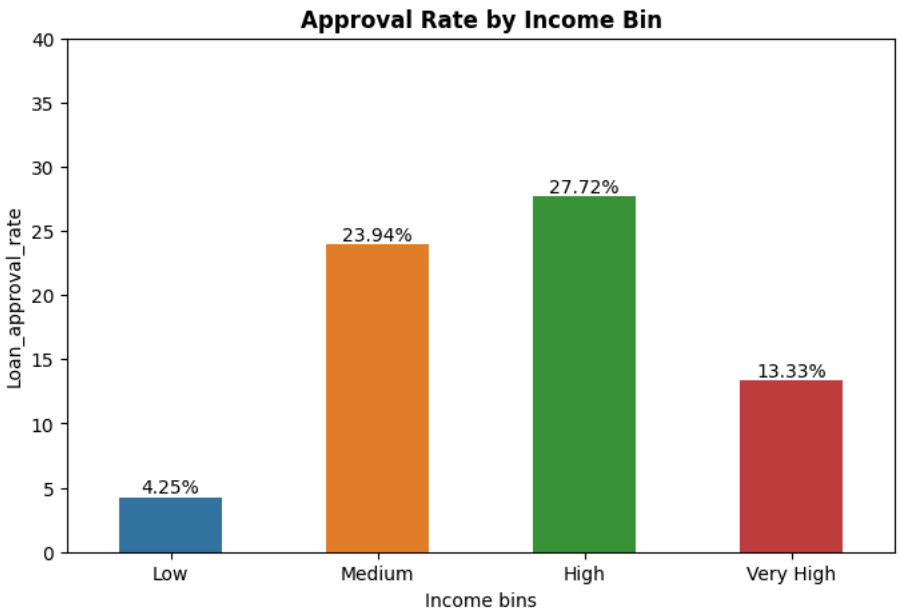
Next steps: Generate code with df New interactive sheet

Approval Rate by Income Bin

```
df_apv_rate = df.groupby('Income_Bin')['LoanApproved'].agg(['count' , 'sum' ])

df_apv_rate['Approval_rate'] = (df_apv_rate['sum']/df_apv_rate['count'])*100

df_apv_rate['Rejection_rate']  =  (1- df_apv_rate['Approval_rate'])*100
```

```
/tmp/ipykernel_1635/324450047.py:1: FutureWarning: The default of observed=False is deprecated and will be changed to True i
  df_apv_rate = df.groupby('Income_Bin')['LoanApproved'].agg(['count' , 'sum' ])
```

```
ax = sns.barplot( data = df_apv_rate  , y = 'Approval_rate'  , x = 'Income_Bin' , hue = 'Income_Bin'  , width = 0.5)
for i in ax.containers:
  ax.bar_label(i , fontsize = 10 , fmt= '%.2f%%' )

plt.title('Approval Rate by Income Bin' , fontweight='bold')
plt.ylabel('Loan_approval_rate')
plt.xlabel('Income bins')

plt.ylim(0 , 40)

plt.show()
```



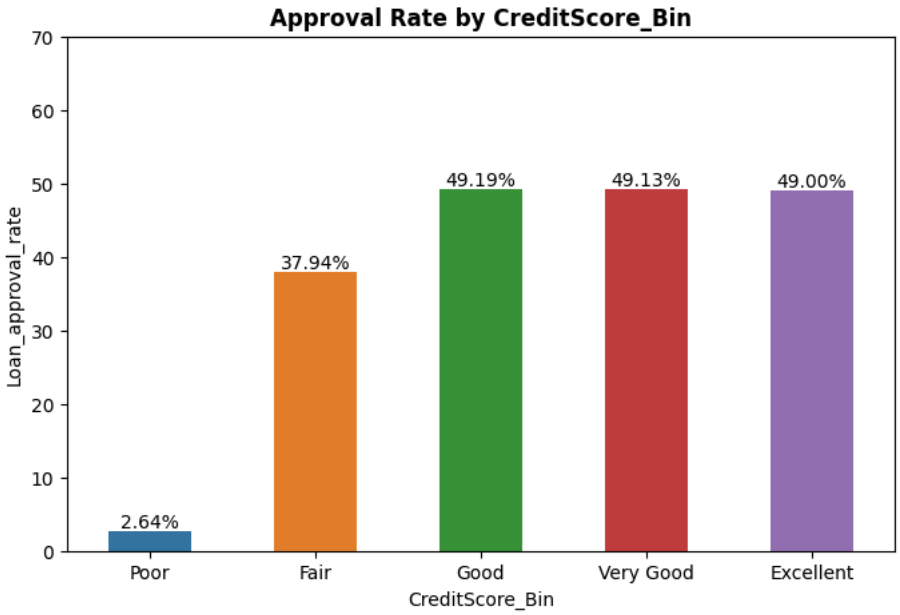
Approval rates increase from low to high income but drop for very high income, suggesting non-linear behavior and influence of other factors

Approval Rate Vs Credit_score

```
df_apv_rate_c = df.groupby('CreditScore_Bin')['LoanApproved'].agg(['count' , 'sum' ])

df_apv_rate_c['Approval_rate'] = (df_apv_rate_c['sum']/df_apv_rate_c['count'])*100

df_apv_rate_c['Rejection_rate']  =  (1- df_apv_rate_c['Approval_rate'])*100
```

```
/tmp/ipykernel_1635/2181562050.py:1: FutureWarning: The default of observed=False is deprecated and will be changed to True
  df_apv_rate_c = df.groupby('CreditScore_Bin')['LoanApproved'].agg(['count' , 'sum' ])
```

```
ax = sns.barplot( data = df_apv_rate_c  , y = 'Approval_rate'  , x = 'CreditScore_Bin' , hue = 'CreditScore_Bin'  , width =
for i in ax.containers:
  ax.bar_label(i , fontsize = 10 , fmt= '%.2f%%' )

plt.title('Approval Rate by CreditScore_Bin' , fontweight='bold')
plt.ylabel('Loan_approval_rate')
plt.xlabel('CreditScore_Bin')

plt.ylim(0 , 70)

plt.show()
```

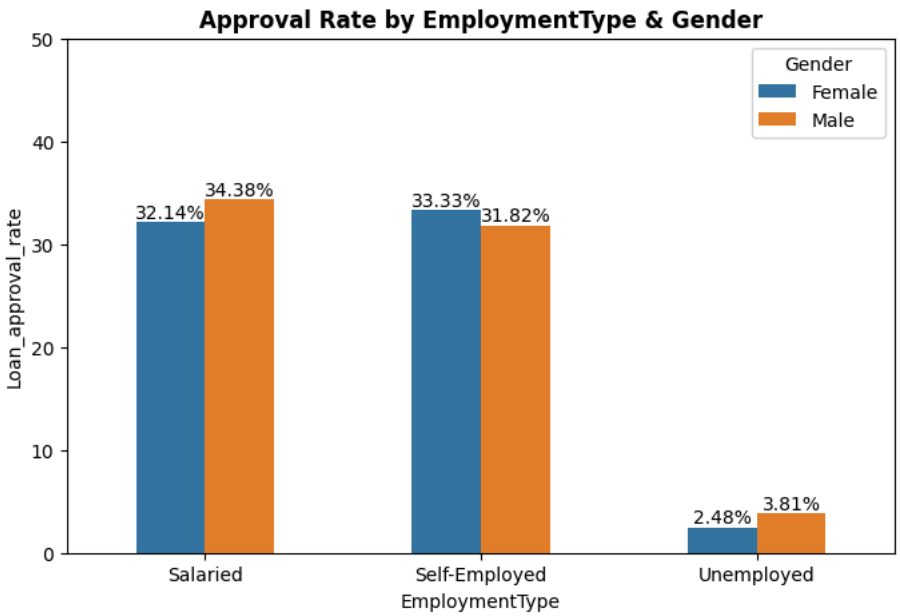


Approval rate sharply increases from ~2% (poor credit) to ~49% (good and above), indicating a clear threshold-based decision model.

Approval Rate by EmploymentType & Gender

```
multi_df = df.groupby(['EmploymentType', 'Gender'])['LoanApproved'].agg(['count' , 'sum' ]).reset_index()

multi_df['Approval_rate'] = (multi_df['sum'] / multi_df['count'])*100
```

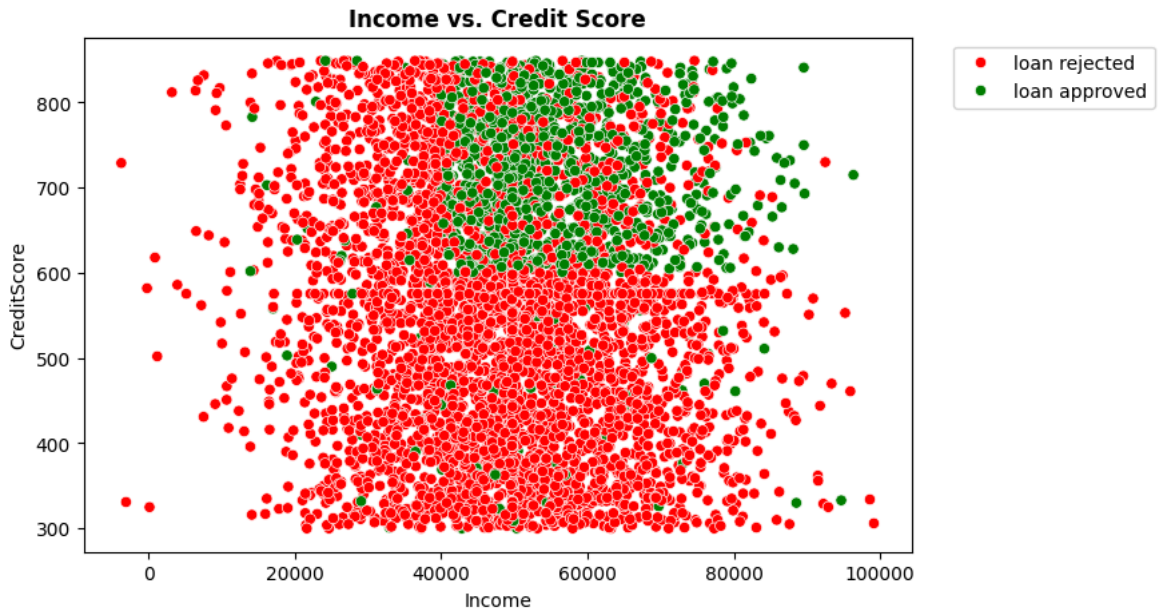
```
ax = sns.barplot(data = multi_df , y = 'Approval_rate' , x = 'EmploymentType' , hue = 'Gender' , width = 0.5)

for i in ax.containers:
  ax.bar_label(i , fontsize = 10  , fmt = '%.2f%%' )

plt.title('Approval Rate by EmploymentType & Gender' , fontweight='bold')
plt.ylabel('Loan_approval_rate')

plt.ylim(0 , 50)

plt.show()
```

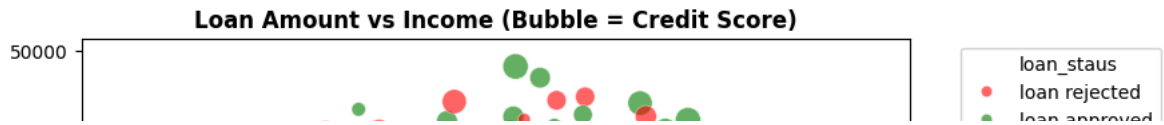


Gender differences are minimal within each employment category, indicating that employment status dominates over gender in influencing approval decisions

Income Vs Credit score Scatter plot

```
sns.scatterplot( data = df , x = 'Income' , y = 'CreditScore' , hue = 'loan_staus'  , palette={{'loan rejected' : 'red' ,
plt.legend( bbox_to_anchor = (1.04 , 1) , loc = 'upper left')

plt.title('Income vs. Credit Score', fontweight='bold')

plt.show()
```



Loan approvals cluster in the high credit score region regardless of income, indicating credit score outweighs income in decision-making

```
from matplotlib.patches import bbox_artist
plt.figure(figsize=(8,6))

sns.scatterplot(
    data=df,
    x='Income',
    y='LoanAmount',
    size='CreditScore',
    hue= 'loan_staus',
    sizes=(20, 200),   # controls bubble size range
    alpha=0.6 , palette={{'loan rejected' : 'red' ,  'loan approved' : 'green'})
)


plt.legend(bbox_to_anchor = (1.05 , 1) , loc = 'upper left')
plt.title("Loan Amount vs Income (Bubble = Credit Score)" , fontweight = 'bold')
plt.xlabel("Income")
plt.ylabel("Loan Amount")

plt.show()
```